

Artificial Intelligence using Python

Hands-on Exercise No.4 Solution

By Bakht Zamin

Fashion_MNIST_CNN

Problem Solutions:

Tasks:

1. Data Loading and Preprocessing

- Load the Fashion-MNIST dataset from keras datasets
- Load the training and test images along with their labels.
- Normalize the pixel values to the range [0, 1].
- Reshape the images appropriately for input to a CNN (e.g., (28, 28, 1) for grayscale images).
- Provide a few sample images with their corresponding labels

Solution:



```
[ ] Load the Fashion-MNIST dataset
[ ] (X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

[ ] Get the Shape of X_train
[ ] X_train.shape
[ ] (60000, 28, 28)

[ ] Get the Shape of X_test
[ ] X_test.shape
[ ] (10000, 28, 28)

[ ] Normalize pixel values
[ ] X_train = X_train / 255.0
[ ] X_test = X_test / 255.0
```

```

Reshape for CNN input

X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)

One-hot encode labels

y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)

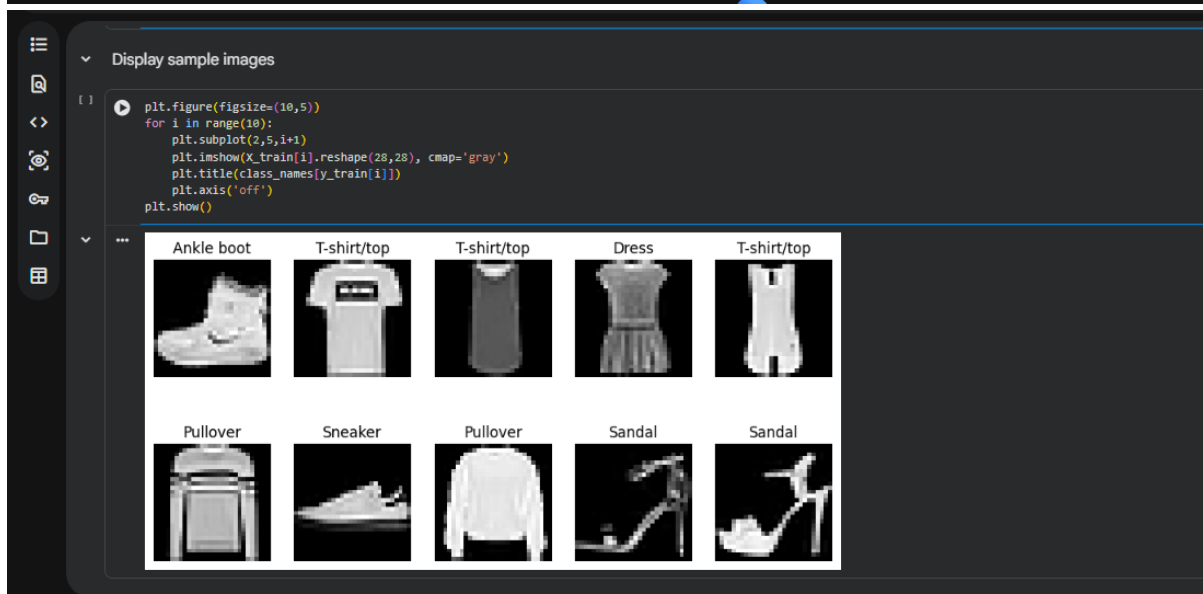
Class names

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

Display sample images

plt.figure(figsize=(10,5))
for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(X_train[i].reshape(28,28), cmap='gray')
    plt.title(class_names[i%trainfil])

```



2. CNN Model Design

- Design a multi-layer Convolutional Neural Network using TensorFlow/Keras
- Include at least:
 - o 2 convolutional layers
 - o MaxPooling layers
 - o Dropout for regularization
 - o Flatten layer before the dense layers
 - o Dense layers leading to the output layer
- Compile the model using an appropriate optimizer, loss function, and evaluation metric for classification

```
Step 2: CNN Model Design

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dropout, Flatten, Dense

model = Sequential()
model.add(Input(shape=input_shape)) # Use Input layer to specify input shape

model.add(Conv2D(filters=32, kernel_size=(3,3), activation='relu'))
model.add(Conv2D(filters=64, kernel_size=(3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

model.add(Conv2D(filters=64, kernel_size=(3,3), activation='relu'))
model.add(Conv2D(filters=128, kernel_size=(3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

model.add(Conv2D(filters=50, kernel_size=(3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

model.add(Flatten())

model.add(Dense(units=1024, activation='relu'))
model.add(Dense(units=128, activation='relu'))

model.add(Dense(units=n_class, activation='softmax'))
```

```
Model Training

model.compile(optimizer=Adam(learning_rate=0.001), loss='categorical_crossentropy', metrics=['acc'])
model.summary()
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
conv2d_19 (Conv2D)	(None, 26, 26, 32)	320
conv2d_20 (Conv2D)	(None, 24, 24, 64)	18,496
max_pooling2d_13 (MaxPooling2D)	(None, 12, 12, 64)	0
conv2d_21 (Conv2D)	(None, 10, 10, 64)	36,928
conv2d_22 (Conv2D)	(None, 8, 8, 128)	73,856
max_pooling2d_14 (MaxPooling2D)	(None, 4, 4, 128)	0
conv2d_23 (Conv2D)	(None, 2, 2, 50)	57,650
max_pooling2d_15 (MaxPooling2D)	(None, 1, 1, 50)	0
flatten_5 (Flatten)	(None, 50)	0
dense_13 (Dense)	(None, 1024)	52,224
dense_14 (Dense)	(None, 128)	131,200
dense_15 (Dense)	(None, 10)	1,290

Total params: 371,964 (1.42 MB)
Trainable params: 371,964 (1.42 MB)
Non-trainable params: 0 (0.00 B)

3. Model Training

- Train the CNN using the training dataset
- Use a validation split (e.g., 10–20%) to monitor the model performance during training
- Train the model for a sufficient number of epochs to achieve stable accuracy
- Display the training and validation accuracy/loss per epoch

Solution:

▼ Step 3: Model Training

```
[ ] h = model.fit(X_train, y_train_cat, validation_split=0.2, epochs=2, verbose=1, batch_size=32, shuffle=True)

▼ ... Epoch 1/2
1500/1500 ————— 213s 140ms/step - acc: 0.0984 - loss: 2.3028 - val_acc: 0.0995 - val_loss: 2.3028
Epoch 2/2
1500/1500 ————— 255s 136ms/step - acc: 0.1000 - loss: 2.3027 - val_acc: 0.0983 - val_loss: 2.3028
```

4. Model Evaluation

- Evaluate the trained model on the unseen test dataset
- Report the test accuracy and loss
- Generate a confusion matrix and classification report showing performance on different fashion categories

Solution:

```
[ ] from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

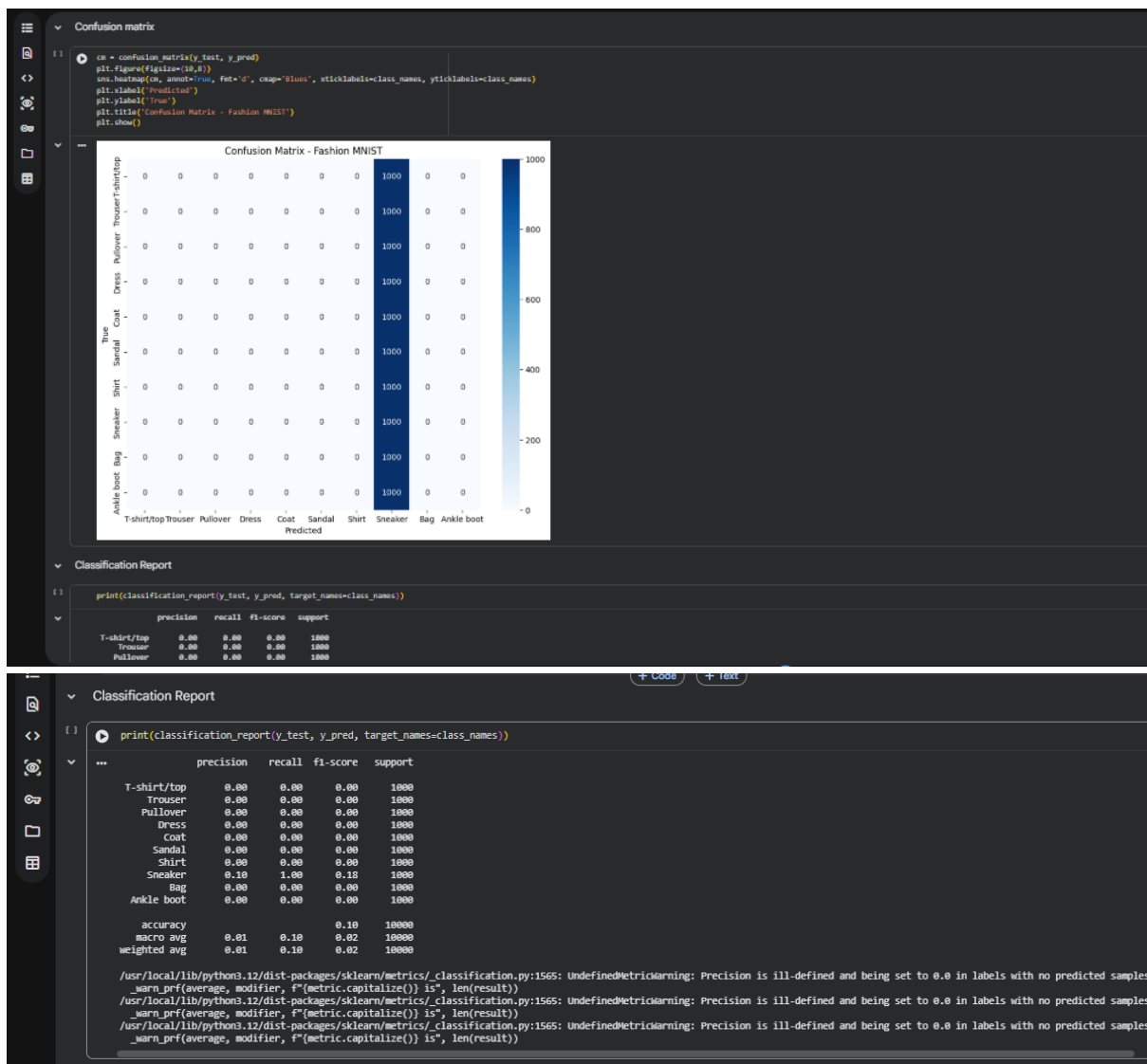
# Evaluate
test_loss, test_acc = model.evaluate(X_test, y_test_cat, verbose=0)
print(f"Test Accuracy: {test_acc:.4f}, Test Loss: {test_loss:.4f}")

▼ Test Accuracy: 0.1000, Test Loss: 2.3027

▼ Predictions

[ ] y_pred_prob = model.predict(X_test)
y_pred = np.argmax(y_pred_prob, axis=1)

▼ 313/313 ————— 12s 37ms/step
```



5. Visualization and Analysis

- Plot the training loss and accuracy curves across epochs
- Plot a few test images along with the predicted and true labels

Solution:

